Instalação e operação (ambiente local)

Documentação técnica - Prumo

Para hospedagem em produção, ver IMPLANTACAO-GOOGLE-CLOUD.md.

Requisitos

- Node.js compatível com o Vite 8 (testado com Node 22 e 24) e npm.
- PostgreSQL (testado com a versão 16) e `psql` para aplicar o schema.
- Opcional: projeto e bucket no Supabase Storage.

1. Banco de dados

Crie o banco e aplique o schema. O script é transacional e pode ser executado de novo sem perder dados.

```
$env:DATABASE_URL='postgres://postgres:senha@localhost:5432/manual_operacional'
psql "$env:DATABASE_URL" -v ON_ERROR_STOP=1 -f backend/sql/schema.sql
```

O próprio `schema.sql` cria a conta inicial (não existe arquivo de seed separado):

- Usuário: `admin`
- Senha: `admin123` (troque no primeiro acesso)

Detalhes das tabelas: Banco de Dados/README.md.

2. Backend

```
cd backend
Copy-Item .env.example .env
npm install
npm run dev
```

Variável	Obrigatória	Descrição
`PORT`	Não	Porta HTTP; padrão `4000`
`DATABASE_URL`	Sim	Conexão PostgreSQL
`JWT_SECRET`	Sim	Segredo de assinatura dos tokens. Sem ele o servidor sobe, mas todo login falha
`FRONTEND_URL`	Sim em produção	Origem permitida pelo CORS; padrão `http://localhost:5173`
`SUPABASE_URL`	Não	URL do projeto Supabase
`SUPABASE_SERVICE_ROLE_KEY`	Não	Chave privada, usada somente no backend
`SUPABASE_BUCKET`	Não	Bucket; padrão `manuals`
`DEV_DEMO_MODE`	Não	`true` usa dados em memória quando o PostgreSQL está indisponível (só desenvolvimento)
`DB_POOL_MAX`	Não	Conexões com o banco por instância do backend; padrão `10`

Variável	Obrigatória	Descrição
`DB_IDLE_TIMEOUT_MS`	Não	Tempo até fechar uma conexão ociosa; padrão `30000`
`DB_CONNECTION_TIMEOUT_MS`	Não	Espera máxima por uma conexão livre ou nova; padrão `5000`
`DB_STATEMENT_TIMEOUT_MS`	Não	O PostgreSQL cancela consultas mais longas que isso; padrão `15000`
`DB_IDLE_TRANSACTION_TIMEOUT_MS`	Não	Encerra transações abertas e paradas; padrão `15000`
`DB_SSL`	Não	`true` quando o banco exige SSL
`TRUST_PROXY_HOPS`	Não	Quantos proxies ficam na frente do backend, para ler o IP real do usuário; padrão `1`
`KEEP_ALIVE_TIMEOUT_MS`	Não	Tempo de conexões keep-alive; padrão `65000`, maior que o do balanceador
`SHUTDOWN_TIMEOUT_MS`	Não	Prazo para terminar requisições em andamento ao desligar; padrão `10000`
`UPLOAD_MAX_PDF_MB`	Não	Tamanho máximo do PDF; padrão `20`. Se mudar, ajuste `frontend/src/constants/uploads.js`
`UPLOAD_MAX_VIDEO_MB`	Não	Tamanho máximo do vídeo; padrão `200`. Se mudar, ajuste `frontend/src/constants/uploads.js`
`STORAGE_DRIVER`	Não	`s3`, `supabase` ou `local`. Sem ele: `s3` se houver `S3_BUCKET`, senão `supabase` se houver credenciais, senão `local`
`SERVE_FRONTEND`	Não	`true` faz o backend entregar também as telas (`frontend/dist`). Usado no piloto num computador só
`S3_BUCKET`	Para `s3`	Nome do bucket (privado)
`S3_ENDPOINT`	Para R2	`https://<ID_DA_CONTA>.r2.cloudflarestorage.com`
`S3_REGION`	Não	`auto` no R2 (padrão)
`S3_ACCESS_KEY_ID` / `S3_SECRET_ACCESS_KEY`	Para `s3`	Token de API do bucket, com leitura e escrita só nele. Segredo: não versionar
`S3_FORCE_PATH_STYLE`	Não	`true` só para emuladores locais (MinIO etc.)
`UPLOAD_URL_TTL_SECONDS`	Não	Validade da URL de envio; padrão `600`
`FILE_URL_TTL_SECONDS`	Não	Validade da URL de leitura; padrão `14400` (4 horas)

Sem Supabase, os arquivos são gravados em `backend/uploads` (pasta ignorada pelo Git). Nunca exponha SUPABASE_SERVICE_ROLE_KEY no frontend.

3. Frontend

```
cd frontend
Copy-Item .env.example .env
npm install
npm run dev
```

Variável	Descrição
`VITE_API_URL`	URL do backend vista pelo navegador; padrão `http://localhost:4000`

Acesse `http://localhost:5173`.

Build de produção

```
cd frontend
npm run build        # gera frontend/dist

cd ../backend
npm start
```

Por padrão o backend **não** serve o `frontend/dist`; ele precisa de um host estático ou proxy reverso. A exceção é o piloto num computador só, abaixo.

Piloto num computador da rede (custo zero)

O sistema roda num computador Windows do hospital e os usuários acessam pelo navegador, dentro da rede interna. O banco fica no Neon (gratuito) e os PDFs e vídeos em `backend\uploads` desse computador.

```
Usuários na rede -> http://<IP do computador>:4000 -> backend (telas, API e arquivos)
                                                       +-> banco no Neon (internet)
```

Uma vez só:

1. Configure o `backend\.env` com o banco do Neon e `STORAGE_DRIVER=local` (seção 2).
2. Peça à TI uma reserva de IP para o computador, para o endereço não mudar.
3. Libere a porta no firewall do Windows (PowerShell como administrador):

```
New-NetFirewallRule -DisplayName "Prumo" -Direction Inbound -Protocol TCP -LocalPort 4000 -Action Allow -
Profile Domain,Private
```

Para ligar o sistema: dê dois cliques em `deploy\pc\iniciar-piloto.bat`. O script gera as telas com `npm run build:piloto`, liga o backend com `SERVE_FRONTEND=true` e mostra o endereço para os usuários (`http://<IP>:4000`). Fechar a janela desliga o sistema.

- No build do piloto as telas chamam a API no mesmo endereço em que foram abertas, então não é preciso configurar IP no `frontend\.env`.
- Para ligar junto com o Windows: tecla Windows + R, digite `shell:startup` e crie ali um atalho para o `iniciar-piloto.bat`.
- Depois de mudar o código, feche a janela e abra o script de novo: ele gera as telas outra vez.

Backup dos arquivos: o banco está no Neon, mas PDFs e vídeos só existem neste computador. Copie a pasta para um compartilhamento da rede todo dia (Agendador de Tarefas do Windows):

```
robocopy "<pasta do projeto>\backend\uploads" "\\SERVIDOR\PASTA\manual-uploads" /E /R:2 /W:5
```

Limitações: o sistema fica fora do ar quando o computador desliga, reinicia ou a janela é fechada; só funciona dentro da rede do hospital; o acesso é por HTTP, sem criptografia; e os links de PDFs e vídeos não expiram. Para sair disso, veja Implantacao-VM-Google.pdf ou Piloto-Gratuito.pdf.

Verificações

```
cd backend
npm test             # política de senha e nota do questionário
node --check src/app.js

cd ../frontend
npm run build
```

Teste de saúde: `GET http://localhost:4000/health` deve responder `{"status":"ok","service":"Prumo API"}`. Esse teste não verifica o banco. Para conferir também o PostgreSQL, use `GET /health/ready`, que responde `503` quando o banco está fora do ar.

Modo demonstração

Com `DEV_DEMO_MODE=true`, se o PostgreSQL estiver fora do ar (conexão recusada, senha errada ou banco inexistente), a API usa dados em memória. Os dados somem ao reiniciar, alguns recursos não funcionam (questionário, painel completo) e o modo **nunca** deve ser ligado em produção.

Diagnóstico rápido

Sintoma	Causa provável
401	Token ausente, inválido ou expirado
403	Perfil sem permissão
"Banco de dados indisponível ou credenciais incorretas"	`DATABASE_URL` errada ou PostgreSQL parado
`secretOrPrivateKey must have a value` no login	`JWT_SECRET` não definido
Erro de CORS no navegador	`FRONTEND_URL` diferente da origem real (protocolo e porta incluídos)
Upload local não abre	Permissões da pasta `backend/uploads`
Progresso do PDF não atualiza	Backend não reiniciado ou `schema.sql` não reaplicado